



การทดสอบซอฟต์แวร์ (Software Testing)

- ❑ ทำหลังจากที่เขียนโปรแกรมเสร็จเรียบร้อยแล้ว
- ❑ เป็นการทดสอบความสมบูรณ์ของโปรแกรม รวมทั้งความน่าเชื่อถือและความถูกต้องของผลลัพธ์จากโปรแกรมที่พัฒนาขึ้น
- ❑ ตรวจสอบประสิทธิภาพการทำงานของซอฟต์แวร์ไปด้วย
- ❑ เป็นผลให้สัมพันธ์กับคุณภาพของซอฟต์แวร์ตามไปด้วย
- ❑ เป็นกิจกรรมที่จัดทำขึ้นเพื่อประเมินและปรับปรุงคุณภาพของซอฟต์แวร์ โดยการหาข้อผิดพลาดและปัญหาที่เกิดขึ้นและทำการแก้ไขปัญหา

Verification

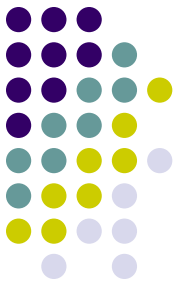


การตรวจสอบว่า ระบบทำงานตามที่กำหนดไว้หรือไม่ ?

การตรวจสอบว่าการสร้างระบบทำอย่างถูกต้องหรือไม่

Are we building the system right ?

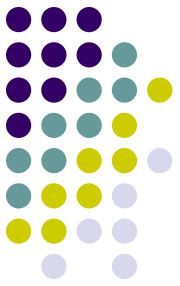
Validation



การตรวจสอบว่า ระบบสามารถทำงานตามความต้องการของ
ผู้ใชหรือไม่ ?

การตรวจสอบว่าระบบที่พัฒนาขึ้นมา นั้นถูกต้องหรือไม่

Are we building the right system ?



Testing & Debug

- **Testing** คือการหาข้อผิดพลาดว่ามีหรือไม่
- **Debug** คือการหาตำแหน่งของข้อผิดพลาด (เพื่อจะได้แก้ไข)

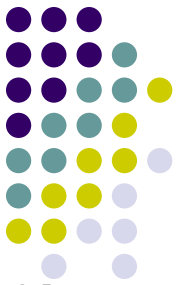
ข้อผิดพลาด(error)



ข้อผิดพลาดที่เกิดขึ้นในการพัฒนาโปรแกรม

- Syntax error เป็นข้อผิดพลาดที่เกิดจากการเขียนโปรแกรมผิด Syntax ของภาษา เกิดขึ้นตอน Compile โปรแกรม
- Logical error เป็นข้อผิดพลาดที่เกิดจากการเขียนโปรแกรมเขียนผิดตรรกะ ทำให้ได้ผลลัพธ์ไม่เป็นไปตามต้องการ เกิดขึ้นตอนโปรแกรมปฏิบัติงาน (execute)
- Runtime error เป็นข้อผิดพลาดที่เกิดขึ้นขณะที่โปรแกรมปฏิบัติงาน (execute) ซึ่งโปรแกรมไม่มี Syntax error แล้ว แต่ยังอาจมี Logical error อยู่ เช่นเกิดจากตัวหารเป็นศูนย์ เป็นต้น

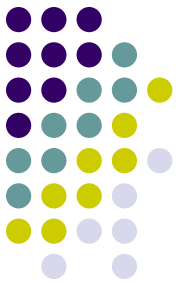
กลวิธีการทดสอบซอฟต์แวร์



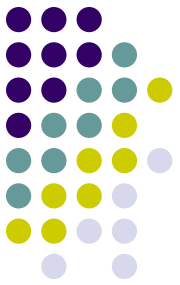
- กลยุทธ์การทดสอบซอฟต์แวร์ – วิธีการออกแบบกรณีทดสอบ (Test case) และการวางแผนทดสอบ (Test Plan) เพื่อให้ได้ชุดของขั้นตอนตามที่ปฏิบัติ เป็นการยืนยันว่าการสร้างซอฟต์แวร์ประสบความสำเร็จ
- กลยุทธ์ใดๆ ต้องมีแผนการทดสอบ การออกแบบกรณีทดสอบ การลงมือทดสอบ และการรวบรวมและประเมินผลข้อมูลผลลัพธ์

หลักการและเป้าหมายของการทดสอบ คือ การหาความผิดพลาดให้พบ
กรณีทดสอบที่ดีควรมีความเป็นได้สูงที่จะหาข้อผิดพลาดพบ

Test case



- บอกถึงสถานการณ์ต่างๆ ที่โปรแกรมต้องตอบสนอง
- ครอบคลุมตั้งแต่ สถานการณ์เริ่มต้น สถานการณ์ต่างๆ ที่มีผลต่อการดำเนินการของโปรแกรมและผลลัพธ์ที่คาดหวัง



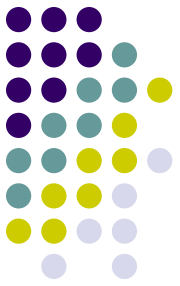
Test case design

- หมายถึง การออกแบบกรณีทดสอบต่างๆ สำหรับการทดสอบโปรแกรม
- การออกแบบกรณีทดสอบเป็นงานที่มีความสำคัญเช่นเดียวกับการออกแบบโปรแกรม
- การออกแบบกรณีทดสอบต้องจัดทำขึ้นเพื่อให้ได้ข้อผิดพลาดมากที่สุด เลวร้ายที่สุดเท่าที่จะพบได้
- กฎที่สำคัญของการออกแบบคือ การออกแบบกรณีทดสอบต่างๆ ต้องครอบคลุมทุกลักษณะของโปรแกรม

ลักษณะของกรณีทดสอบที่ดี



- A good test has a high probability of finding an error -- มีความน่าจะเป็นสูงในการหาความผิดพลาดพบ
- A good test is not redundant. – ไม่ซ้ำซ้อน
- A good test should be “best of breed” -- ควรเป็นตัวที่คิดมาดีที่สุด
- A good test should be neither too simple nor too complex – ไม่ธรรมดาเกินไปหรือซับซ้อนเกินไป



เทคนิคการทดสอบโปรแกรม

แบ่งออกเป็น 2 กลุ่มใหญ่คือ

- Manual Testing
 - การทดสอบโดยไม่ใช้เครื่องคอมพิวเตอร์
- Automated Testing
 - การทดสอบด้วยเครื่องคอมพิวเตอร์

Manual Testing

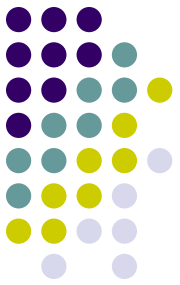


- เป็นการทดสอบโปรแกรมโดยการตรวจสอบของโปรแกรมเมอร์เอง ไม่มีการใช้เครื่องคอมพิวเตอร์ แบ่งได้เป็น 2 ชนิดคือ

1) Inspection

- โปรแกรมเมอร์ตรวจสอบเอง
- เปรียบเทียบ Code ของโปรแกรมที่เขียนขึ้นกับ รายการ Error ที่ทราบแล้ว
- เป็นการทดสอบความผิดพลาดของ Code เท่านั้น
- ป้องกันข้อผิดพลาดรูปแบบเดิมไม่ให้เกิดซ้ำอีก
- ไม่ทำให้ทราบว่าผลลัพธ์ถูกต้องหรือไม่

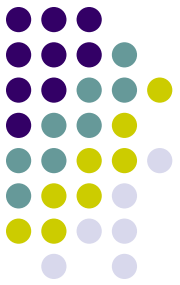
Manual Testing



2) Desk Checking

- การตรวจสอบโค้ดของโปรแกรมตามลำดับคำสั่งว่ามีตรรกะผิดพลาดหรือไม่
- ทำโดยผู้ที่ได้รับการแต่งตั้งให้เป็นผู้ทดสอบ
- วิธีการนี้ไม่เหมาะสำหรับงานที่ความซับซ้อนสูงเนื่องจากเสียเวลามากในการทดสอบ

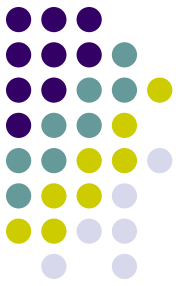
Automated Testing



แบ่งได้เป็น 5 ชนิดคือ

- Syntax checking
- Unit testing/Module Testing
- Integration testing
- Stub testing
- System testing

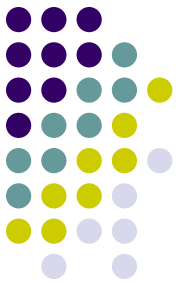
Automated Testing



Syntax Checking

- ตรวจสอบไวยากรณ์ที่เขียนขึ้นโดยใช้ Compiler
- ใช้เวลาไม่นาน
- ไม่สามารถตรวจสอบผลลัพธ์ได้

Automated Testing

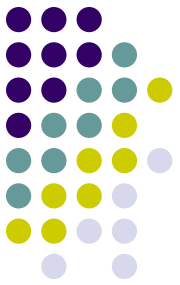


Unit Testing

- เรียกอีกชื่อว่า Module Testing
- โปรแกรมเมอร์ทดสอบ Module ที่ตนเองรับผิดชอบ
- หาข้อผิดพลาดในการทำงานแต่ละ โมดูล

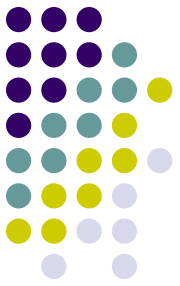
มีใช้ 2 แบบคือ

- Black Box Testing
- White Box Testing



Black Box Testing

- เป็นการทดสอบ โดยไม่คำนึงถึงคำสั่งภายในโปรแกรม
- เป็นการทดสอบ Function ต่างๆ ของโปรแกรมตาม Requirements ที่มี
- เป็นการทดสอบ โดยดูค่า Output จาก Input ที่ให้กับโปรแกรม ต้องมีความสอดคล้องกัน
- สนใจเฉพาะว่าระบบมี Function การทำงานอย่างไร และจะได้ output ออกมาเป็นอะไรบ้าง



Black Box Testing

- การกำหนดข้อมูลในการทดสอบ ได้แก่
 - ค่าตัวแทนของกลุ่ม
 - ค่าสูงสุด
 - ค่าต่ำสุด
 - ค่าเกินพิกัด
 - ค่าที่ผิดปกติ

Black Box Testing

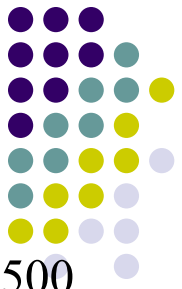


สำหรับ Technique ต่างๆ ที่ใช้ในการทดสอบแบบ Black box testing นั้นมีหลายวิธี แต่จะขอยกตัวอย่างที่รู้จักกันดี คือ

- **Equivalence partitioning**

- เป็นการกำหนดค่าตัวแทน ของกลุ่มข้อมูลขึ้นมา 1 ค่า แล้วนำค่านี้มาใช้ในการทดสอบ ซึ่งจะสามารถ Apply ได้ว่าถ้าใช้ค่าตัวแทนนี้มาทำการทดสอบได้ผลลัพธ์อย่างไร ค่าอื่นๆ ที่อยู่ภายใต้กลุ่มนี้ ก็จะมีผลลัพธ์เช่นเดียวกัน

Black Box Testing



เช่น ระบบธนาคารสามารถให้โอนเงินผ่าน ATM ขั้นต่ำคือ 100 และสูงสุดคือ 500 บาท

การทดสอบจะต้องกำหนดตัวแทนของกลุ่มของข้อมูลที่จะต้องนำมาทดสอบ ตัวอย่างเช่น

กรณีโอนเงิน 50 บาท

(เป็นตัวแทนของกลุ่มที่ < 100)

กรณีโอนเงิน 200 บาท

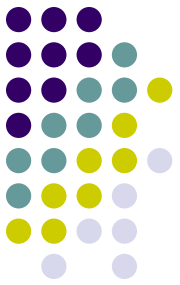
(เป็นตัวแทนของกลุ่มที่อยู่ระหว่าง
100 - 500)

กรณีโอนเงิน 1000 บาท

(เป็นตัวแทนของกลุ่มที่ > 500)

*** เพราะฉะนั้นจะได้ชุดของข้อมูลมาสามชุด และได้ data มาทดสอบ 3 ค่าเท่านั้น

Black Box Testing



- **Boundary value analysis**

- เป็นวิธีการทดสอบโดยกำหนดขอบเขตของข้อมูลขึ้นมา เพื่อจะได้ค่า input data ที่ครอบคลุม
- เช่น ระบบธนาคารสามารถให้โอนเงินผ่าน ATM ขั้นต่ำคือ 100 และ สูงสุดคือ 500 บาท
- การทดสอบจะต้องกำหนดขอบเขตของข้อมูลที่จะต้องนำมาทดสอบ

Black Box Testing

- ตัวอย่างเช่น

กรณีโอนเงิน 99 บาท

กรณีโอนเงิน 100 บาท

กรณีโอนเงิน 101 บาท

กรณีโอนเงิน 499 บาท

กรณีโอนเงิน 500 บาท

กรณีโอนเงิน 501 บาท



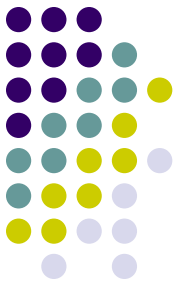
ตัวอย่าง:: การทดสอบแบบกล่องดำ



- หน้าจอการรับค่าข้อมูล User และ Password โดยลูกค้าให้สเปคมาว่า User ต้องเป็นชื่อภาษาอังกฤษไม่ต่ำกว่า 5 แต่ไม่เกิน 8 ตัวอักษร และ Password ก็เช่นกัน ควรจะมีการทดสอบอย่างไรเพื่อทำให้ไม่เกิดข้อผิดพลาดกับระบบ

A screenshot of a 'Log in' dialog box. The title 'Log in' is centered at the top. Below it, there are two input fields. The first is labeled 'User:' and the second is labeled 'Password:'. Both fields are empty and have a yellow border. At the bottom, there are two buttons: 'OK' and 'CANCEL', both with a gray background and black text.

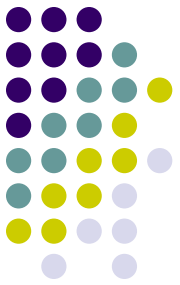
การออกแบบกรณีทดสอบ (Test Case)



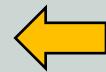
- Input:
 - User และ Password
- Condition
 - $5 \leq \text{ความยาว User} \leq 8$
 - $5 \leq \text{ความยาว Password} \leq 8$
- Output:
 - Pass, Fail

การออกแบบกรณีทดสอบ

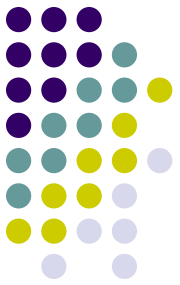
- ชุดทดสอบ (Test case set)



Test ID	User	Password	Expected Result	Actual Results	Accepted Results
1	aaa	bbb	Fail	Fail	Pass
2	aaaa aaaa	bbbbbbb b	Pass		
3	aa	bbb	Fail		
4	aaa	bb	Fail		
5					
6					

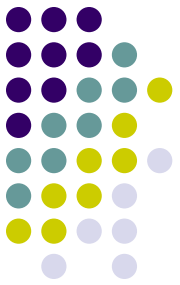


Test case



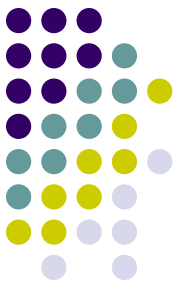
White Box Testing

- เป็นการทดสอบเพื่อดูโครงสร้างของโปรแกรม หรือทางเดินในโปรแกรม
- ต้องสร้างชุดทดสอบเฉพาะสำหรับทดสอบในเงื่อนไขต่างๆ โดยชุดทดสอบจะต้องประกอบด้วยชุดที่สามารถประมวลผลอย่างปกติและไม่ปกติ
- ต้องมีความรู้เรื่องของระบบว่ามีการออกแบบอย่างไร ทำงานอย่างไร
- ต้องมีความรู้ในเรื่องของ Programming



Basis Path Testing

- เป็น idea ที่ช่วยในการทำ White-box ให้ง่ายขึ้น
- เปลี่ยน Flowchart ให้เป็น Graph ที่ประกอบด้วย Vertices และ Edge
- เพื่อทดสอบความสลับซับซ้อนทางตรรกะวิทยาโดยจะทดสอบทุกๆ “Execution path” การออกแบบกรณีทดสอบจะทำการทดสอบ “Basic set” ที่ใช้ “Execute logical path” ต่างๆ คือ
 - 1) Sequence logical path
 - 2) If logical path
 - 3) Loop logical path



Basis Path Testing

- การทดสอบแบบ Basis Path จะต้องรู้จำนวนเส้นทางความซับซ้อนของปัญหา (cyclomatic complexity หรือแทนด้วยกราฟ $V(G)$) โดย $V(G)$ สามารถหาได้จาก 3 วิธี ดังนี้

วิธีที่ 1.

$V(G)$ = จำนวนของพื้นที่แบบปิด (enclosed area) ของผังงานโปรแกรม +1

วิธีที่ 2

$V(G)$ = จำนวนของสัญลักษณ์ตัดสินใจในผังงานโปรแกรม (simple decisions) +1

วิธีที่ 3

$V(G)$ = Edge (จำนวนเส้นเชื่อมระหว่าง Node) – Node (จำนวน Node ภายในกราฟ) +2

Basis Path Testing

วิธีที่ 1.

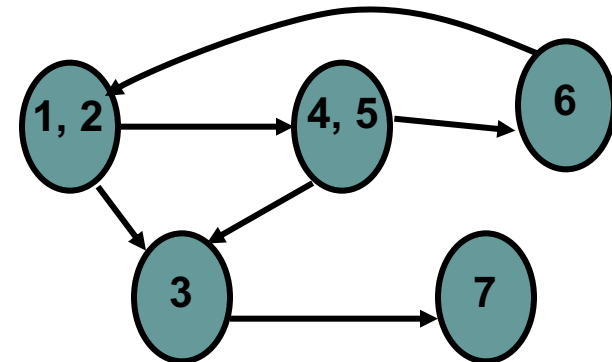
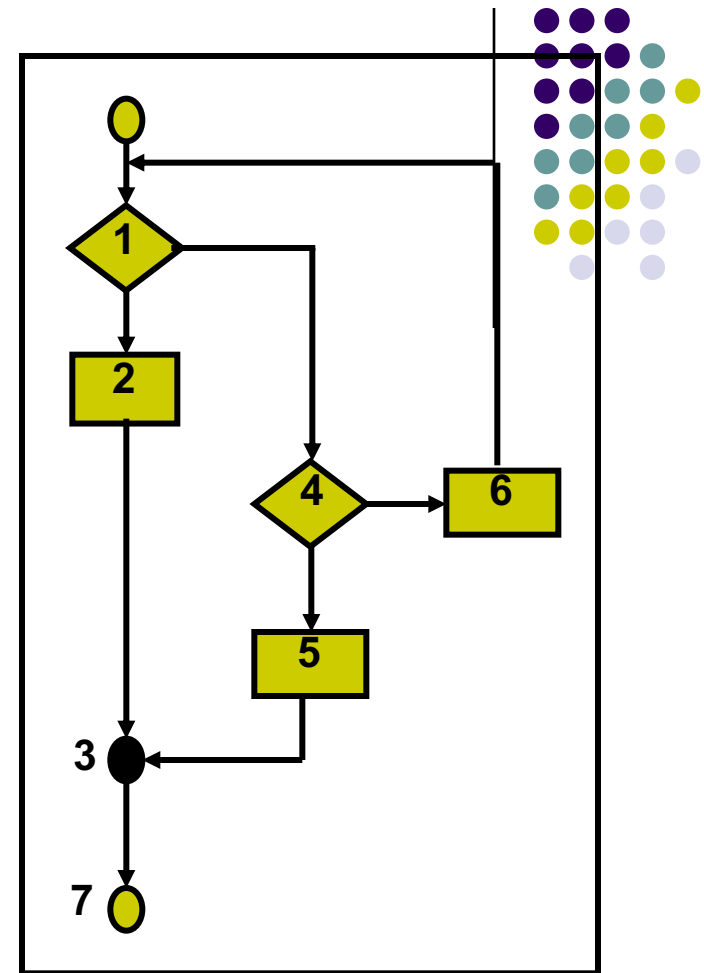
$V(G)$ = จำนวนของพื้นที่แบบปิด (enclosed area) ของ
ผังงานโปรแกรม + 1 ; $V(G) = 2 + 1 = 3$

วิธีที่ 2

$V(G)$ = จำนวนของสัญลักษณ์ตัดสินใจในผังงาน
โปรแกรม (simple decisions) + 1 ; $V(G) = 2 + 1 = 3$

วิธีที่ 3

$V(G)$ = Edge (จำนวนเส้นเชื่อมระหว่าง Node) –
Node (จำนวน Node ภายในกราฟ) + 2 ; $V(G) = 6 - 5$
+ 2 = 3



พิจารณาเส้นทางที่แยกเป็นอิสระ (independent path)



- เส้นทางอิสระ (independent path) เป็นเส้นทางใดๆ ผ่านโปรแกรมที่มีประโยชน์คำสั่งหรือประโยชน์ทดสอบเงื่อนไขอย่างน้อย 1 ประโยค
- จำนวนเส้นทางที่แยกเป็นอิสระ ได้จากค่า $V(G)$

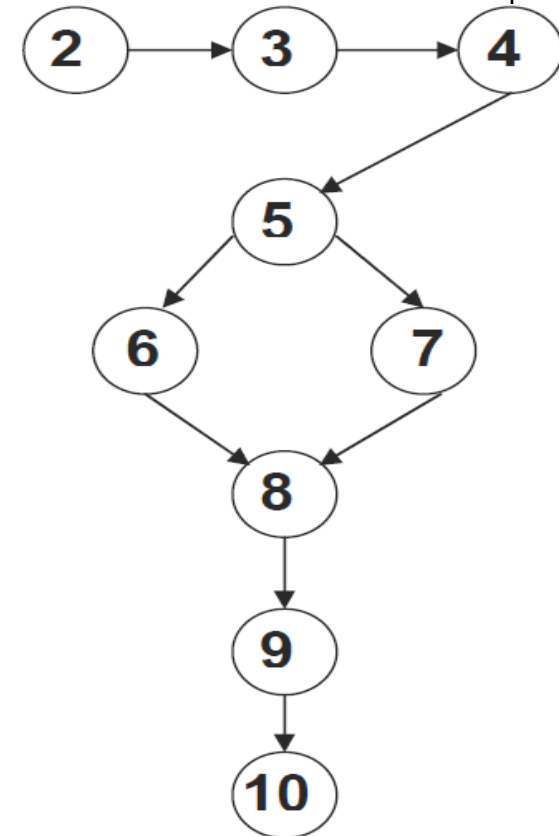
การสร้างกรณีทดสอบ (test case)



- สร้างกรณีทดสอบ (test case) ที่จะนำมาทดสอบในแต่ละ path เลือกข้อมูลให้มีค่าเหมาะสมในการทดสอบแต่ละเส้นทาง
- เมื่อกำหนดกรณีทดสอบให้กับทุก path แล้วให้ทำการทดสอบแต่ละ path และเปรียบเทียบผลลัพธ์ที่ได้ กับ ผลลัพธ์ที่คาดหวัง (Expected results)

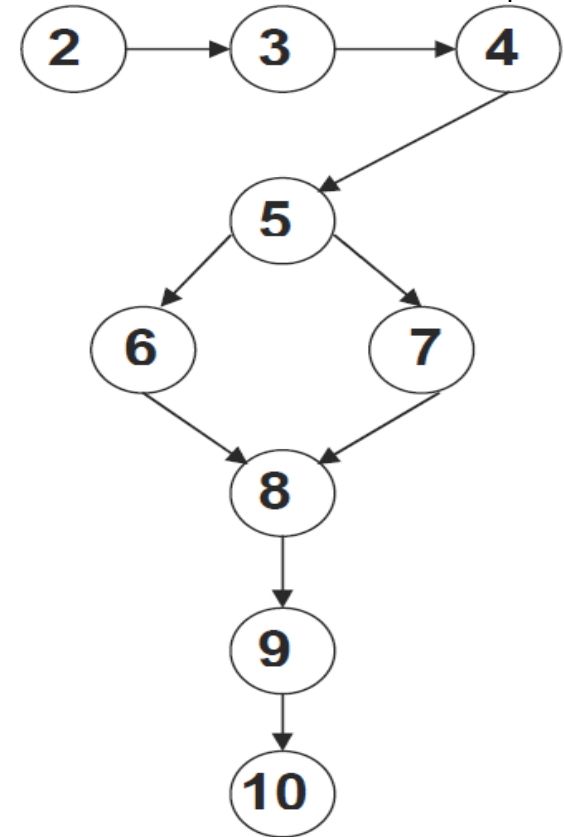
Path Testing

1. Program 'Simple Subtraction'
2. Input (x, y)
3. Output (x)
4. Output (y)
5. If $x > y$ then DO
6. $x - y = z$
7. Else $y - x = z$
8. EndIf
9. Output (z)
10. Output "End Program"



หาจำนวน Path ในการทดสอบ

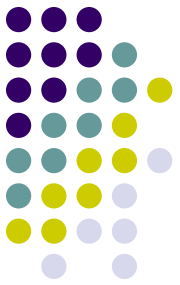
- หาจำนวน Path
- $V(G) = \text{Edge} - \text{Node} + 2$
- $V(G) = 9 - 9 + 2 = 2$



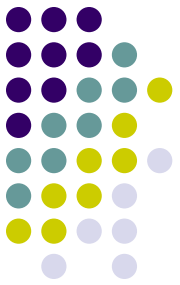
การสร้างกรณีทดสอบ (test case)

Path 1: 2-3-4-5-6-8-9-10

Path 2: 2-3-4-5-7-8-9-10



Test ID	x	y	Expected Results (Z)
1	2	1	1
2	1	2	1



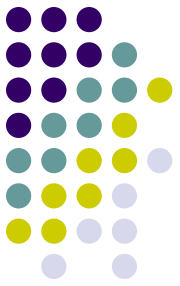
Loop Testing

- เป็นเทคนิคในการทดสอบแบบ White-Box โดยมุ่งประเด็นไปที่ความถูกต้องของโครงสร้างของการวนซ้ำหรือลูป (loop constructs)

มีกลุ่มของลูปที่แตกต่างกัน มีอยู่ 4 แบบ ได้แก่

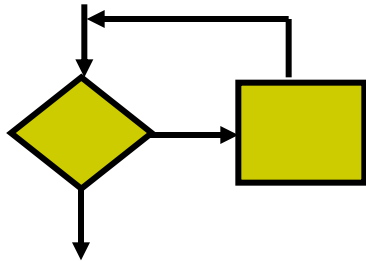
1. ลูปอย่างง่าย
2. ลูปซ้อนลูป
3. ลูปต่อกัน
4. ลูปไม่เป็นโครงสร้าง

Loop Testing

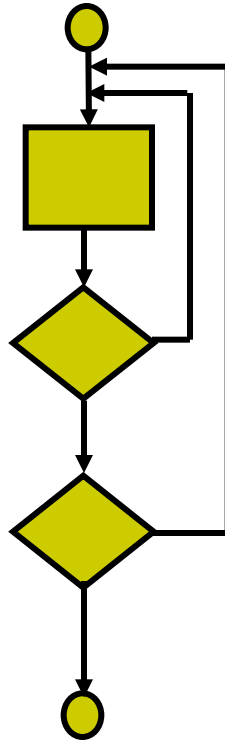


- | | |
|-----------------------|---|
| 1. Simple Loop | เป็นรูปแบบง่ายไม่มีความซับซ้อน |
| 2. Nested loop | เป็นลูปที่มีโครงสร้างลักษณะนำมาซ้อนกัน
ภายในมีความซับซ้อนมากขึ้น |
| 3. Concatenated loop | เป็นลูปที่มีการนำมาต่อกันเป็นโครงสร้างใหญ่ขึ้น |
| 4. Unstructured loops | เป็นลูปที่ไม่มีโครงสร้างของลูปชัดเจนมีความ
ซับซ้อนการทดสอบมีความยุ่งยากมาก |

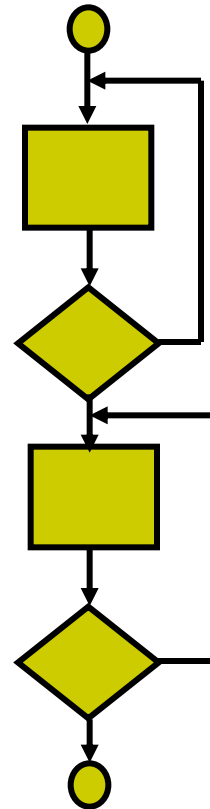
Loop Testing



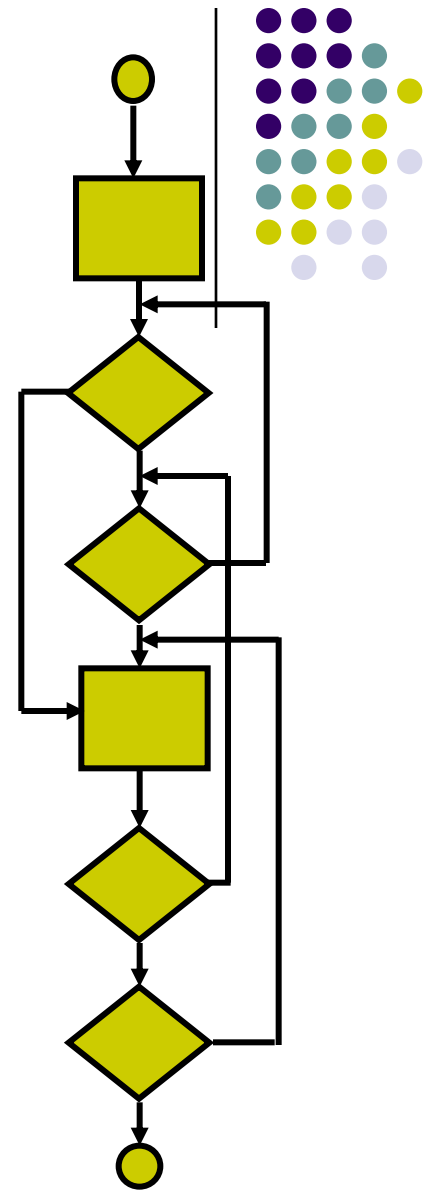
Simple loops



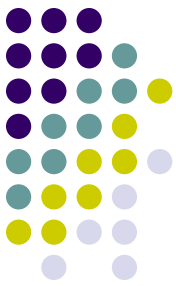
Nested loops

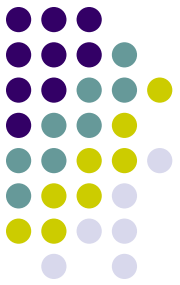


Concatenated loops



Unstructured loops

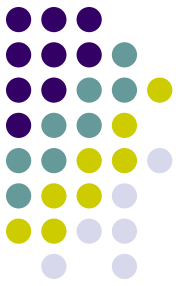




Loop Testing

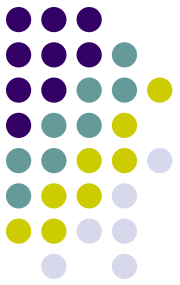
- ชุดทดสอบลูปอย่างง่าย เมื่อ n เป็นจำนวนรอบสูงสุดที่อนุญาตให้ผ่านลูป
 1. ทำงานข้ามลูปไปเลย
 2. ทำงานผ่านลูปเพียงหนึ่งรอบ
 3. ทำงานผ่านลูป 2 รอบ
 4. ทำงานผ่านลูป m รอบ เมื่อ $m < n$
 5. ทำงานผ่านลูป $n-1, n, n+1$ รอบ

Loop Testing



ในการทดสอบแบบลูปซ้อนลูป จะทำให้การทดสอบเพิ่มขึ้นเป็นตามจำนวนชั้นของลูปที่ซ้อน มีข้อแนะนำในการทดสอบ

- เริ่มทดสอบจากลูปในสุดก่อน โดยตั้งค่าลูปอื่นๆ ให้มีค่าน้อยที่สุด
- ทำการทดสอบจากลูปในไปลูปนอกทีละชั้น
- ทำต่อไปจนกว่าทุกๆ ลูปได้ทดสอบหมด



Automated Testing

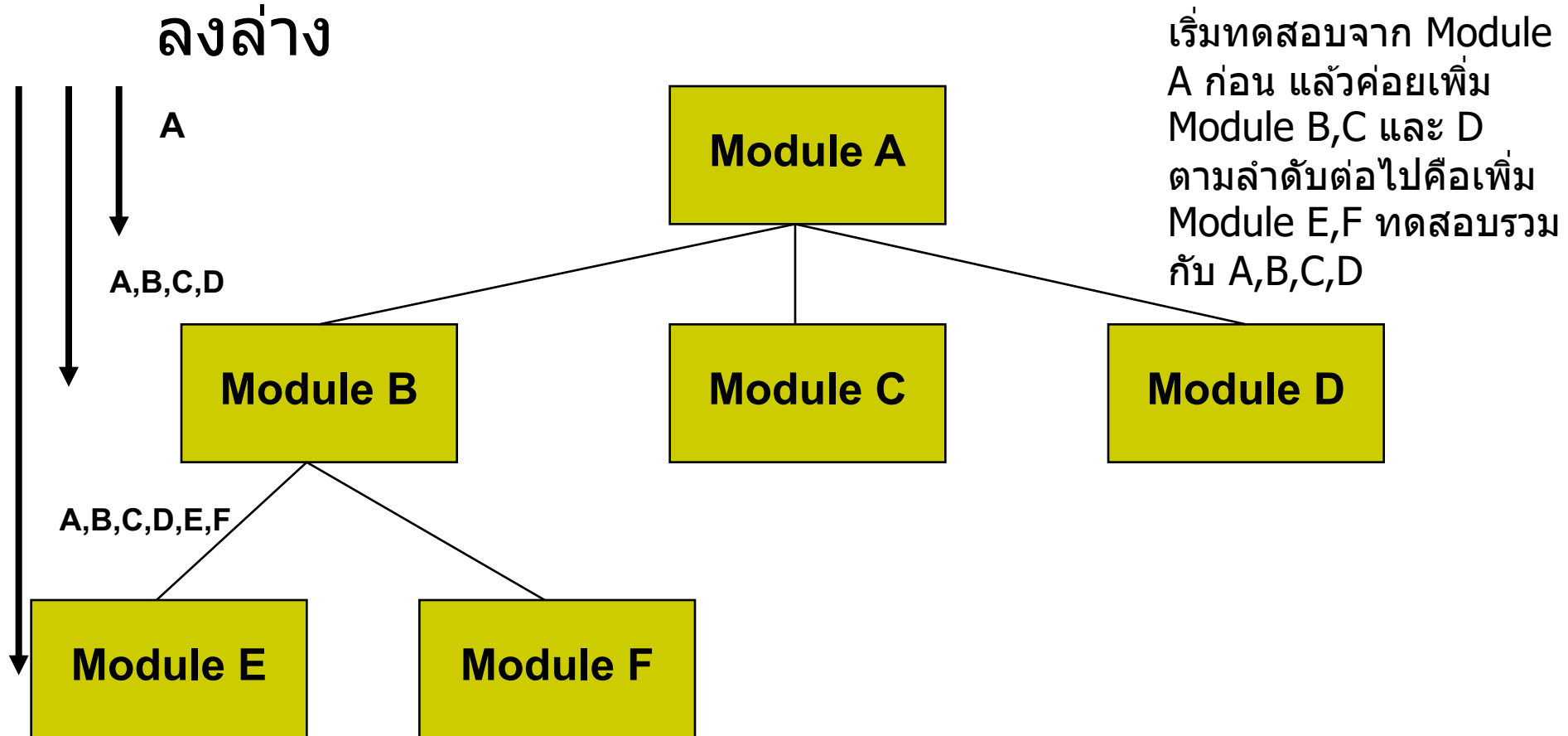
Integration Testing

- เพื่อตรวจสอบความถูกต้องของ Function การทำงานต่างๆ เมื่อมีการ Integrate unit / module เข้าร่วมกัน
- ทดสอบการทำงานรับส่งข้อมูลกันระหว่างโมดูล
- อาศัย structure chart มี 2 แบบ คือ
 - แบบบนลงล่าง (Top-down Approach)
 - แบบล่างขึ้นบน (Bottom-up Approach)

Integration Testing แบบจากบนลงล่าง (Top-down Approach)



- เป็นการทดสอบโปรแกรมโดยทดสอบโมดูลจากบนลงล่าง



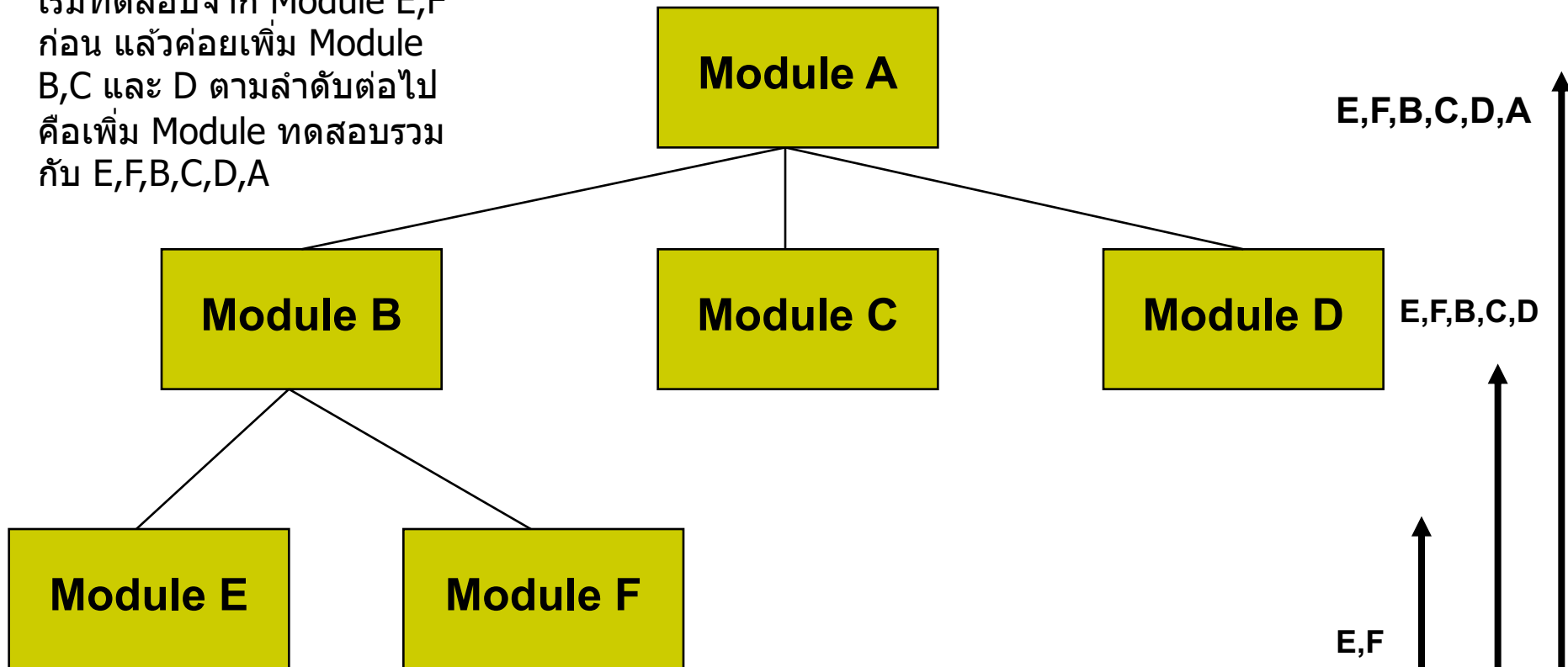
Integration Testing แบบจากล่างขึ้นบน

(Bottom-up Approach)



- เป็นการทดสอบโปรแกรมโดยทดสอบโมดูลล่างขึ้นบน

เริ่มทดสอบจาก Module E,F ก่อน แล้วค่อยเพิ่ม Module B,C และ D ตามลำดับต่อไป คือเพิ่ม Module ทดสอบรวมกับ E,F,B,C,D,A



Automated Testing



Stub Testing

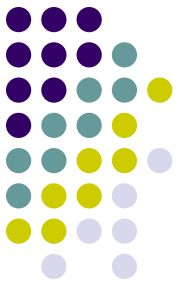
- การทดสอบแบบเพิ่มโมดูล (Integration Testing) นั้นจะค่อย ๆ เพิ่มโมดูลเข้ามาทดสอบตามลำดับ กรณีที่เพิ่มจากบนลงล่าง คือ เริ่มทดสอบจาก Module A ก่อน แล้วค่อยเพิ่ม Module B,C และ D ตามลำดับก่อนเพิ่ม Module E,F แต่ความจริง Module B ต้องเรียกใช้ข้อมูลจาก Module E,F ที่อยู่ในระดับล่างซึ่งยังไม่ถึงรอบการทดสอบ
- ปัญหาคือ อาจไม่เห็นผลลัพธ์ ดังนั้นจึงต้องสร้าง Stub Testing เป็นตัวแทน Module E,F เพื่อทดสอบชั่วคราว
- Stub คือ กลุ่มคำสั่งสั้นๆ ที่เขียนขึ้นมาเพื่อเป็น โมดูลตัวแทนในการทดสอบโปรแกรม

Automated Testing



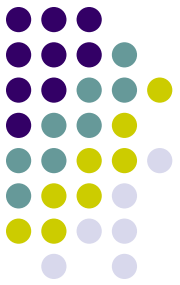
System Testing

- คล้ายการทดสอบแบบ Integration แต่จะต่างกันตรงที่จะทดสอบโปรแกรมหนึ่ง และเพิ่มเรื่อยๆจะครบทุกโปรแกรมของระบบงาน
- เป็นการตรวจสอบการทำงานทั้งระบบ ว่าโปรแกรมทุกโปรแกรมเมื่อทำงานร่วมกันแล้วจะให้ผลลัพธ์ที่ถูกต้องแล้วหรือไม่
- จุดประสงค์ เพื่อตรวจสอบระบบว่าระบบทำงานได้ถูกต้องและได้ผลลัพธ์ตรงตาม Requirement



กลยุทธ์ในการทดสอบประสิทธิภาพของระบบ

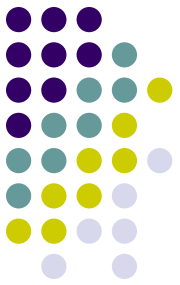
1. **Peak Load Testing**
2. **Performance Testing**
3. **Recovery Testing**
4. **Storage Testing**
5. **Procedure Testing**
6. **User Testing**



กลยุทธ์ในการทดสอบประสิทธิภาพของระบบ

1) การทดสอบการทำงานสูงสุด (Peak Load Testing) เป็นการทดสอบประสิทธิภาพในการประมวลผลของระบบ เมื่อมีการทำรายการมากที่สุด ณ ช่วงเวลาใดเวลาหนึ่ง เพื่อทดสอบว่าระบบรองรับการทำรายการคำสั่งมากที่สุด และนานเท่าไรเมื่อต้องประมวลผลจำนวนรายการคำสั่งมากที่สุด ดังกล่าวในช่วงเวลาหนึ่ง

กลยุทธ์ในการทดสอบประสิทธิภาพของระบบ



2) การทดสอบประสิทธิภาพของเวลา (Performance Testing) เป็นการทดสอบระบบเพื่อพิจารณาถึงช่วงเวลาที่ใช้ในการประมวลผลรายการ ว่าใช้ระยะเวลานานเพียงใดในการทำรายการไม่ว่าจะเป็นการประมวลผลแบบกลุ่ม Batch Processing หรือประมวลผลแบบออนไลน์ (On – Line Processing)



กลยุทธ์ในการทดสอบประสิทธิภาพของระบบ

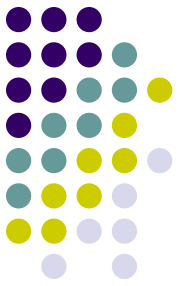
3) การทดสอบการกู้ระบบ (Recovery Testing)

เป็นการทดสอบความสามารถในการกู้ระบบกรณีที่ระบบล้ม รวมถึงความสามารถในการกู้ข้อมูลด้วย

4) การทดสอบการเก็บข้อมูล (Storage Testing) เป็น

การทดสอบว่า เก็บข้อมูลได้สูงสุดเป็นจำนวนเท่าใด เพื่อจะได้เตรียมการรองรับจำนวนข้อมูลที่อาจเพิ่มมากขึ้นในอนาคต

กลยุทธ์ในการทดสอบประสิทธิภาพของระบบ



5) การทดสอบกระบวนการ (Procedure Testing)

เป็นการทดสอบการจัดทำคู่มือการดำเนินงานของระบบ และคู่มือการใช้งานว่าสามารถสร้างความเข้าใจให้กับผู้ใช้หรือไม่ และสามารถใช้เมื่อเกิดปัญหาหรือไม่

6) การทดสอบผู้ใช้ (User Testing) เป็นการทดสอบ

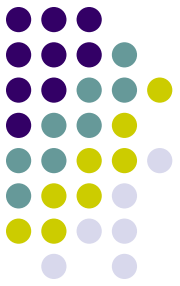
การใช้งานจริงของระบบเพื่อต้องการทราบว่าปัญหาในการใช้ระบบเป็นอย่างไรบ้าง



การทดสอบการยอมรับของผู้ใช้

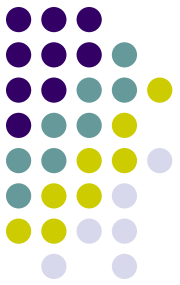
- หลังจากที่ได้ทดสอบความสมบูรณ์และความถูกต้องของโปรแกรมเรียบร้อยแล้ว
- มีขั้นตอนที่มีความสำคัญไม่น้อยไปกว่าการทดสอบโปรแกรม นั่นก็คือ การทดสอบการยอมรับจากผู้ใช้
- เนื่องจากการพัฒนาระบบนั้น ก็เพื่อต้องการตอบสนองความต้องการในการดำเนินงานของผู้ใช้ระบบ
- โดยวิธีการทดสอบการยอมรับของระบบนั้นสามารถแบ่งออกเป็น 2 ประเภทคือ

การทดสอบการยอมรับของผู้ใช้



1) Alpha Testing

- การทดสอบความสมบูรณ์ของระบบโดยผู้ใช้พร้อมกับจะมี Tester/ QA เป็นคนแนะนำ
- ผู้ใช้จะทดสอบระบบขณะยังไม่ได้ติดตั้งในสถานที่จริง โดยทดสอบภายในสถานการณ์จำลองที่กำหนดขึ้น โดยทีมงานพัฒนาระบบ ทีมงานจะบันทึกข้อผิดพลาดและทำการแก้ไข
- ทำให้ได้รู้ว่าระบบมีข้อผิดพลาดอะไรเกิดขึ้นบ้าง



Alpha Testing

มีการทดสอบ 4 ประการคือ

1. **Recovery Testing** เป็นการทดสอบการกู้ระบบ
2. **Security Testing** เป็นการทดสอบความปลอดภัยของระบบ
3. **Stress Testing** เป็นการทดสอบประสิทธิภาพการทำงานของระบบภายใต้ความกดดัน
4. **Performance Testing** เป็นการทดสอบประสิทธิภาพการทำงานของระบบภายใต้สภาพแวดล้อมของคอมพิวเตอร์

การทดสอบการกู้คืน (Recovery Testing)



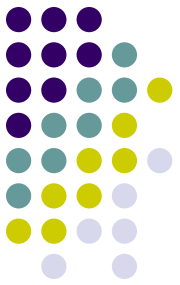
- ความสามารถในการกู้คืนเมื่อระบบเกิดความล้มเหลว
- Fault Tolerance ระบบต้องทำงานต่อได้และต้องทนต่อความผิดพลาด
- การทดสอบ ทำให้ระบบล้มเหลวในสถานการณ์ต่างๆแล้วดูการกู้คืนของข้อมูล

การทดสอบการรักษาความปลอดภัย (Security Testing)



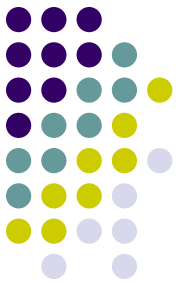
- ❑ มีเครื่องมือในการรักษาความปลอดภัยของระบบเป็นที่น่าสนใจหรือไม่
- ❑ การรักษาความปลอดภัยอย่างง่ายได้แก่ การใช้วิธีการให้สิทธิและการควบคุมสิทธิ
- ❑ การเข้ารหัสข้อมูล

การทดสอบแรงตึงเครียด (Stress Testing)



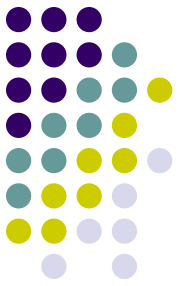
- ❑ เป็นการทดสอบในสถานการณ์ที่ไม่เป็นปกติ เพื่อดูความทนทานในระบบ
- ❑ แรงตึงเครียดของระบบอาจจะเกิดจากการนำเข้าข้อมูลが多เกินไป การประมวลผลที่บ่่อยเกินไป
- ❑ Sensitivity Testing

การทดสอบสมรรถนะ (Performance Testing)



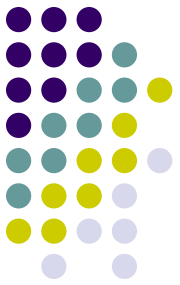
- ❑ เป็นการวัดสมรรถนะของระบบว่าอยู่ในระดับที่ยอมรับได้
- ❑ เช่น ระยะเวลาในการตอบสนองของงาน การจัดสรรพื้นที่หน่วยความจำ
- ❑ การสร้าง Log File เพื่อเป็นการจดบันทึกเพื่อวัดสมรรถนะการทำงานของระบบในแต่ละช่วงเวลา

การทดสอบการยอมรับของผู้ใช้



2) Beta Testing

- การทดสอบความสมบูรณ์ของระบบโดยผู้ใช้ และใช้ข้อมูลจริงในการทดสอบและภายใต้สถานการณ์ที่เกิดขึ้นจริงโดยไม่มีทีมงานไป ฝ้าสังเกต
- ต้องมีคนคอยจดบันทึกข้อผิดพลาดให้กับทีมงาน
- การทดสอบประเภทนี้ถือว่าการซ้อมติดตั้งระบบเพื่อใช้งานจริง
- เนื่องจากเป็นการทดสอบระบบอย่างสมจริงไม่ว่าจะเป็น สถานการณ์ ข้อมูล ขั้นตอนการดำเนินงาน เอกสารคู่มือ การฝึกอบรม การสนับสนุน การทำงาน รวมทั้งเป็นการแก้ปัญหาที่เกิดขึ้นระหว่าง Alpha Testing อีกด้วย

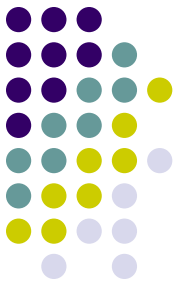


Test Planning : การวางแผนการทดสอบระบบ

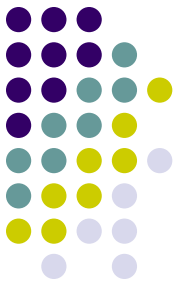
- การกำหนดมาตรฐานของกระบวนการในการทดสอบ เพื่อ
 - ครอบคลุม Errors
 - Minimum Cost
- กำหนดข้อตกลงเบื้องต้นและรายละเอียดของระบบ
- เตรียมแผนงานการทดสอบเพื่อการยอมรับระบบ
- นำข้อมูลการออกแบบมาใช้ในการวางแผนการทดสอบความสัมพันธ์ของระบบรวม
- กำหนดแผนการทดสอบความสัมพันธ์ของระบบย่อย

Test Planning ประกอบด้วย

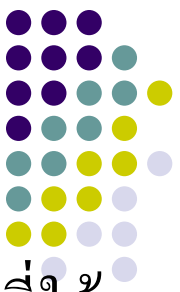
- Scope of Testing
- Test Plan
- Test Procedure
- Actual Test Result



Test Planning



- Scope of Testing สรุปรายละเอียดของข้อกำหนดการทดสอบ ได้แก่ หน้าที่(functional) ประสิทธิภาพ และคุณสมบัติภายใน
- Test Plan อธิบายยุทธวิธีทั้งหมดที่เกี่ยวข้องกับการนำองค์ประกอบย่อย (module, sub-system) แต่ละส่วนมาทดสอบร่วมกัน, ระยะทดสอบ (Test Phase), หมายกำหนดการทดสอบ, สภาพแวดล้อมและทรัพยากรที่ใช้ทดสอบ รวมทั้งอุปกรณ์ ฮาร์ดแวร์และเครื่องมือที่ใช้ในการทดสอบ



Test Planning

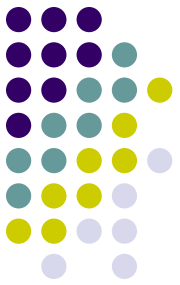
- Test Procedure อธิบายกรรมวิธีทดสอบ ตัวอย่างรายการข้อมูลที่ใช้สำหรับทดสอบโมดูลต่างๆ และผลลัพธ์ที่คาดการณ์ที่เกิดจากการทดสอบ
- Actual Test Result คือส่วนที่บันทึกรายละเอียดผลการทดสอบ, ปัญหา, เงื่อนไขพิเศษที่ปรากฏในขณะทดสอบโปรแกรม

หลังจากวางแผนการทดสอบแล้ว กระบวนการทดสอบจะเริ่มดำเนินการในลักษณะแบบย้อนหลังตามลำดับ เริ่มจากการทดสอบส่วนย่อย Unit testing, การทดสอบการทำงานร่วมกัน integration testing, การทดสอบระบบ system testing , การทดสอบเพื่อการยอมรับ Acceptance testing ก่อนส่งมอบซอฟต์แวร์ให้กับลูกค้า



วิธีการประเมินผลการทำงานของระบบ

- การใช้แบบสอบถาม
- การบันทึกเทปการทำงานของผู้ใช้
- การสร้างส่วนพิเศษภายในระบบ ให้สามารถบันทึกข้อมูลเกี่ยวกับการทำงานของผู้ใช้
- การสร้างระบบให้ผู้ใช้สามารถบันทึกความคิดเห็นของตนเองกำลังใช้งานระบบนั้น ๆ



Who Test Software ?

- Developer
- Independent tester
- Customer

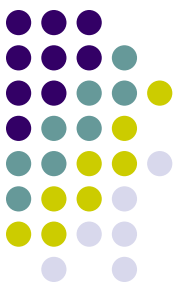


กระบวนการ Debugging

กระบวนการแก้ไขข้อบกพร่องที่ปรากฏขึ้นภายหลังการทดสอบโปรแกรม เริ่มต้นด้วยการนำกรณีทดสอบมาทดลองประมวลผล เพื่อค้นหาสาเหตุของข้อบกพร่องที่ปรากฏในโปรแกรม โดยการเปรียบเทียบผลลัพธ์จากการทดลองกับผลลัพธ์ที่คาดหวัง

ถ้าผลลัพธ์ไม่ตรงกันแสดงว่าพบสาเหตุของข้อบกพร่อง จากนั้นผู้ทดสอบจะค้นหาจุดที่ทำให้เกิดข้อบกพร่อง และทำการแก้ไขเพื่อจัดข้อบกพร่องนั้นทิ้ง

กรณีที่ผลลัพธ์ของการเปรียบเทียบตรงกัน ผู้ทดสอบจะออกแบบกรณีทดสอบใหม่ จนกระทั่งพบข้อบกพร่องในโปรแกรม



กระบวนการ Debugging

จำแนกแนวทางการ debugging ไว้ 3 กลุ่ม คือ

- **Brute force** เป็นวิธีธรรมดาไม่มีประสิทธิภาพในการแยกเหตุการณ์ของการเกิดข้อผิดพลาดในโปรแกรม
- **Backtracking** เหมาะสำหรับโปรแกรมขนาดเล็ก โดยเริ่มแกะรอยการประมวลผลของโปรแกรม ตั้งแต่ตำแหน่งที่ไม่เกิดอาการ จนพบจุดของการเกิดข้อผิดพลาดในโปรแกรม
- **Cause elimination** นำข้อมูลที่ทำให้เกิดข้อผิดพลาดมาแบ่งเป็นส่วนๆ เพื่อทำการทดสอบโดยตั้งสมมติฐานจากเหตุการณ์ที่ทำให้เกิดข้อผิดพลาด และนำตัวอย่างข้อมูลมาทดสอบเพื่อตัดเส้นทางที่ไม่เกี่ยวข้องทิ้ง